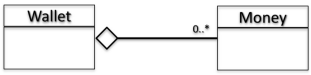

G
O
D
O
O
O
O
D

**Association** - indicates relationships between classes through their objects

**Aggregations** - a special type of association which is one way relationship
- 'has a' relationship between classes
- The existence of the objects (enclosing and enclosed) are independent




Wallet without money 의미 없.

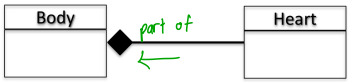
- A "has a" relationship: *Wallet has Money*
- Independence: However, money does not necessarily need to have Wallet.
- Money objects can be created even though there is no Wallet object.

An aggregation relationship is implemented by objects contain pointer to other objects.

? the existence of the enclosing and enclosed objects are independent.

For example, destroying an object will not be destroying the other object.
Only the link / pointing to the object is disconnected

**Compositions** - a restricted version of aggregation in which the enclosing and enclosed objects are highly dependent to each other.




- A "whole-part" relationship: *human Body "consists of"/"has" Heart, or Heart "is part" of human Body.*
- Dependence: A human body needs heart to live and a heart needs a human body to survive. If one dies then another one too.
- If the Body object is destroyed then Heart object also will be gone.
- When a Body object is created, the Heart object is also created.

One object (contained object) becomes part of the other object (container object)
Deleting the container object will also delete the contained object.

**Inheritance** - provides a way to create a new class from an existing class.

classes can inherit attributes and method from other classes
@ add extra

**Purpose** — Specialisation - Extending the functionality of an existing class.
specific entities from the same class but with their own data.
|
Generalisation
(apply Encapsulation)
sharing commonality between two or more class

sharing same structure, different data

Inheritance establish "is-a" relationship between class.

**Access Specifiers** - Controls accessibility to members for each class

Public : object of derived class can be treated as object of base class

Protected : more restrictive, but allows derived class to know details of parent

Private : prevent object of derived class from being treated as object of base class.

UTM

# Accessibility vs. Ownerships

- An object owns all members from the class it was created from, regardless of `private`, `public` or `protected`.
- However, the object can only access to the `public` members.
- A class can access all its own members.

object 모든 멤버
access public 멤버

# Chapter 8 - Polymorphism

↳ ability of objects to perform the same action differently.

Same method , different behaviours

/ use the same name (including parameters) for the method

\ code in the method definition different one to another

Overloaded Method : 2 or more methods share the same name but different parameter.
(- data type / - no. of parameter.)

Redefined Method : derived classes define methods with exactly the same name and parameters as used in the base class.
↑ similar

Overridden methods : dynamically bound (Virtual methods) - the binding is decided at run time

Non-virtual method : the binding is decide at compile time (static binding) - constant / unchange

↑ 除了 overriden methods, 其他 method 都是 static

# Chapter 9

Exceptions - indicate that something unexpected has occured or been detected

① Exceptions - object or value that signals an error

② Throw an exception - send a signal that error has occured

③ Catch / Handle an exception - interpret the signal.

Keyword : throw, try, catch

```
}
//code in the try-block is called protected code
//code in the catch-block is called exception handler
```

- **throw**: send a signal that an error has occurred.
- **try**: followed by a block { }, is used to invoke code that throws an exception.
- **catch**: followed by a block { }, is used to detect and process exceptions thrown in preceding **try** block. Takes a parameter that matches the type thrown.

implement together.

Function Template

Class Template

```
template <class T>
class Grade
{
    private:
        T score;
    public:
        Grade(T);
        void setGrade(T);
        T getGrade()
};
```

if it change the value int/double/...

Function templates can be overloaded — perform same task with different parameter data types

each template must have a unique parameter list

Unlike functions, classes are instantiated by supplying the type name (int, double, string, etc) at object definition

```
classname <int> object1 (1,2);
classname<double> object2 (3.5,4.5);
```

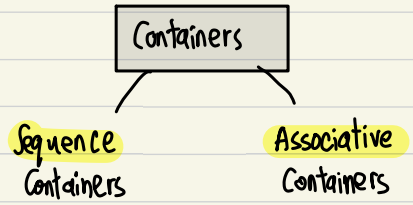
Use to create function / class that can operate on different data type

# Standard Template Library (STL)

- a library containing templates for frequently used data structures and algorithms

Containers

iterators




- organize and access data sequentially

- use keys to allow data to be quickly accessed.

- as in array.

tmr

read, MCQ, class diagram, pass year.

## Size and Flexibility

- Arrays have a fixed size, defined at compile time.
- Vectors have a dynamic size, and can grow or shrink at run time.

(Values may be added to or removed from the end or middle of a vector.)

## Initialization

- Arrays can be initialized with a fixed set of values at compile time.
- Vectors can be initialized with a list of values or resized and assigned value, at run time.

Operations :

- Array do not have build in method for insertion, deletion or resizing.
- Vectors have many build in method such as push_back, pop_back, insert, erase and resize.